

SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING, KALAVAKKAM
(An Autonomous Institution, Affiliated to Anna University, Chennai)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
III Year CSE (V Semester)
Academic Year 2026-27
UCS3513 SYSTEM DESIGN LABORATORY

Lab Exercise 2-Design and Implement a Basic URL Shortener Using Spring Boot

Objective

To design and implement a basic URL Shortener application using Spring Boot that generates a short URL for a given long URL using a hashing technique, and redirects users to the original URL.

Problem Statement

1. Accept a long URL from the user.
2. Generate a short code using a hashing algorithm (MD5 or SHA-256).
3. Use the first 6–8 characters of the generated hash as the short code.
4. Check the database to ensure that the generated short code is unique. If the short code already exists for a different URL, generate a new unique short code (for example, by increasing the hash length or appending a random character).
5. Store the mapping between the long URL and the short code in the database.
6. Redirect the user to the original URL when the generated short URL is accessed.

Tasks

- Design the architecture of the URL Shortener system.
- Design the database table to store URL mappings.
- Implement the following REST APIs:
 - **POST /shorten** – Accept a long URL and generate a unique short URL using hashing.
 - **GET /{shortCode}** – Redirect the user to the corresponding long URL.
- Ensure that each generated short code is **unique** before storing it in the database.
- Test the application using Postman and a web browser.

Sample Input

```
{  
  "longUrl": "https://www.google.com/search?q=system+design"  
}
```

Sample Output

```
{  
  "shortUrl": "http://localhost:8080/aB12Xy"  
}
```

When the generated URL is opened in a browser, it should redirect to:

<https://www.google.com/search?q=system+design>

Expected Learning Outcomes

After completing this experiment, students will be able to:

- Understand the basic workflow of a URL Shortener.
- Design a simple client–server architecture using Spring Boot.
- Develop REST APIs using Spring Boot.
- Use Spring Data JPA to store and retrieve data.
- Generate unique short codes.
- Implement URL redirection.
- Test REST APIs using Postman.